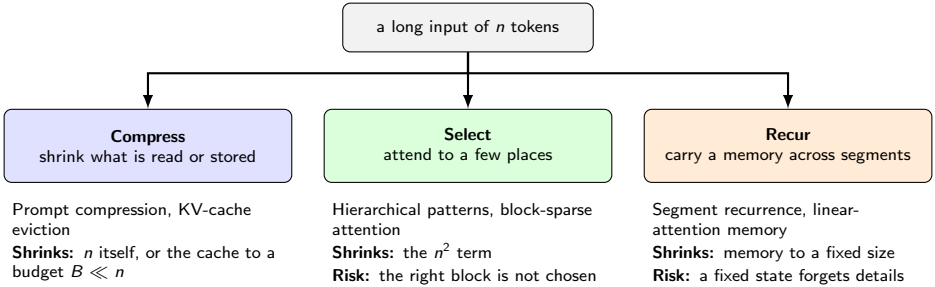
What a Long Context Is For

- Many real inputs do not fit in a few thousand tokens: a code repository and the issue filed against it, a contract or a book, hours of transcribed audio, and the trace an agent accumulates as tool calls pile up.
- Context windows have grown from 2K tokens to 128K and beyond. The Gemini 1.5 report goes further: **near-perfect retrieval (>99%) up to at least 10M tokens** in its experiments.
- A long window lets a model use material it was never trained on. Given a grammar manual for Kalamang, a language with fewer than 200 speakers, Gemini 1.5 translated English to Kalamang at a level similar to a person who learned from the same materials.
- The window is a capacity, and an expensive one. Three questions organise this lecture: what a long context costs, how much of it a model actually uses, and which techniques make long inputs affordable.

Gemini Team, Google, "Gemini 1.5: Unlocking Multimodal Understanding Across Millions of Tokens of Context," 2024, arXiv:2403.05530v5, abstract.

Three Families of Techniques



- Each family trades exactness for cost in a different place. The lecture first measures how much context models really use, then takes the three families in turn and ends with how to choose between them and retrieval.

Learning Outcomes

By the end of this lecture you should be able to:

- **Compute** the attention FLOPs and KV-cache memory of a long context for a given model, and name which of the three costs a technique reduces.
- **Explain** the gap between a claimed and an effective context length, and **choose** benchmarks that expose it.
- **Compare** prompt-compression methods and KV-cache eviction policies, and predict what each one loses.
- **Explain** hierarchical attention, from fixed local and global patterns to learned block selection, and **compute** its cost.
- **Derive** segment-level recurrence and linear-attention memory, and state what a fixed-size memory can and cannot recall.
- **Choose** between a longer window, retrieval, compression and memory for a given workload.
- **Prerequisite knowledge:** the KV cache and its quantization, GQA and MLA, FlashAttention; RoPE rescaling, sliding-window attention; RNNs, state-space models and their copying limit, mixture of experts; soft prompts, LoRA, RAG.

Claimed Versus Effective Context Length

- A model's **claimed** context length is the longest input it accepts, n tokens. It states a capacity; how much of it the model uses has to be measured.
- RULER, a benchmark of 13 synthetic tasks, defines the **effective** context length as the longest tested length whose score still clears a pass mark:

$$n_{\text{eff}} = \max\{\, n \in \mathcal{N} : a(n) > \tau \,\},$$

  where $a(n)$ is the mean accuracy over the tasks at length n, $\mathcal{N}$ the set of tested lengths (4K to 128K) and τ the pass mark.
- RULER sets $\tau = 85.6\%$, the score of Llama-2-7B, a 4K-context model, at 4K. In its words, "almost all models fall below the threshold before reaching the claimed context lengths."
- n_{eff} belongs to a benchmark and a threshold; quote both with the number.

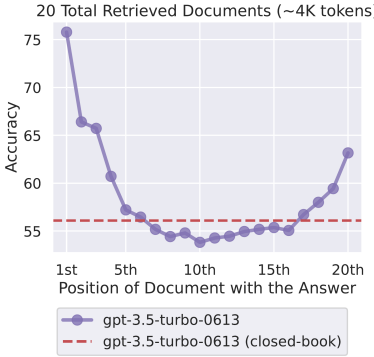
Model	Claimed	Effective	4K	128K
Gemini-1.5-Pro	1M	>128K	96.7	94.4
GPT-4	128K	64K	96.6	81.2
Llama3.1 (70B)	128K	64K	96.5	66.6
Llama3.1 (8B)	128K	32K	95.5	77.0
Yi (34B)	200K	32K	93.3	77.3
GradientAI/Llama3 (70B)	1M	16K	95.1	72.1
LWM (7B)	1M	<4K	82.3	65.0

Mean accuracy (%) over the 13 tasks at 4K and 128K tokens. LWM (7B) starts below the pass mark at 4K, so its effective length is under 4K although it degrades slowly.

Hsieh et al., "RULER: What's the Real Context Size of Your Long-Context Language Models?" COLM 2024, arXiv:2404.06654v3, Sections 1 and 4, Table 3.

Lost in the Middle

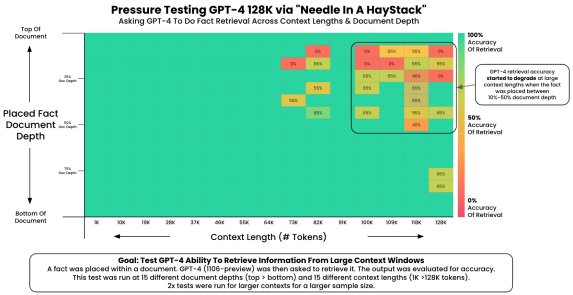
- Liu et al. measured the edge bias under controlled conditions. Each prompt holds p Wikipedia passages of at most 100 tokens: one answers a NaturalQuestions-Open question, and the other $p - 1$ are the passages a retriever ranks highest among those without the answer. Only the position of the answering passage changes.
- GPT-3.5-Turbo with $p = 20$ is right 75.8% of the time with the passage first, 63.2% with it last, and 53.8% at position 10: a U-shaped curve of primacy and recency.
- With no passages (closed book) it scores 56.1%; with only the answering passage, 88.3%. At position 10 it does worse than with no passages at all.
- A longer window does not change the curve. GPT-3.5-Turbo (16K) and Claude-1.3 (100K) score almost exactly like their 4K and 8K versions on inputs that fit in both.
- What to report: accuracy as a function of position, and the gap between the best and the worst position. An average over positions hides the middle.



Accuracy against the position of the answering passage; the dashed line is closed-book accuracy. Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," TACL 2024, arXiv:2307.03172v3, Figure 1; numbers from Tables 1 and 6.

The Needle-in-a-Haystack Test

- **Needle in a haystack (NIAH)**, released by Greg Kamradt in November 2023: insert one sentence, the needle, at depth x% of a haystack of Paul Graham essays cut to n tokens, ask for it, and sweep x and n into a heatmap.
- Needle: "The best thing to do in San Francisco is eat a sandwich and sit in Dolores Park on a sunny day." Question: "What is the best thing to do in San Francisco?" ("most fun thing" in the code released in November 2023). An LLM judge scores the answer from 1 to 10.
- It tests literal retrieval of one fact. On GPT-4 128K, accuracy "started to degrade at large context lengths when the fact was placed between 10%–50% document depth".
- A pass is weak evidence: the question repeats the needle's words, there is one fact and nothing to combine, and recent models score near-perfectly.
- Models with "nearly perfect accuracy in the vanilla NIAH test" still show "large performance drops as the context length increases" on RULER. Across 59 models, HELMET finds that "synthetic tasks like NIAH do not reliably predict downstream performance."

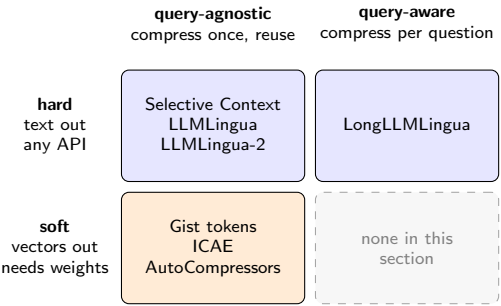


Source: G. Kamradt, `needle-in-a-haystack` repository, github.com/gkamradt/needle-in-a-haystack (`img/GPT_4_testing.png`), run of 8 Nov. 2023, MIT licence.

Quotes: Hsieh et al., arXiv:2404.06654v3, abstract; Yen et al., "HELMET: How to Evaluate Long-Context Language Models Effectively and Thoroughly," ICLR 2025, arXiv:2410.02694v3, abstract.

A Map of Prompt Compression

- Prompt compression shortens the input itself: fewer tokens, or a few vectors in place of many tokens. The model then runs as usual on the shorter input, so it stacks with the other techniques.
- The **compression ratio** is $n/\tilde{n}$, where n is the number of tokens in the original prompt and $\tilde{n}$ the number of tokens or vectors the LLM reads instead. LLMLingua cuts a 2,366-token prompt to 117 tokens (next slide), a ratio of about 20×.
- **Hard** methods output text: here a subset of the original tokens, though paraphrasing methods also count as hard. A **black-box** model, reachable only through a text API, can read it, and a person can audit it.
- **Soft** methods output vectors the LLM reads in place of tokens, as input embeddings or as keys and values at every layer. They need the weights and some training.
- **Query-aware** methods see the question and keep what it needs; **query-agnostic** methods compress once and reuse the result for any question.



Two independent choices: what the compressor outputs, and whether it sees the question. Every soft method here compresses without the question, so its vectors can be computed once and cached.

Hard/soft split after Li et al., "Prompt Compression for Large Language Models: A Survey," NAACL 2025, aclanthology.org/2025.naacl-long.368.

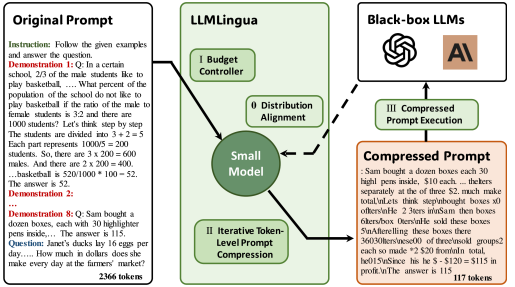
Deleting Low-Information Tokens

- **Selective Context** scores each token with a small causal LM by its **self-information**

$$I(x_t) = -\log_2 p(x_t \mid x_{<t}),$$

where x_t is the t-th prompt token and p the small model's next-token probability. Predictable tokens carry little information. Perplexity is 2 raised to the mean of I.

- Token scores are summed over noun phrases (self-information is additive) and phrases below a chosen percentile are deleted. Deleting half cut per-token time from 110.8 to 76.3 ms in the paper's Vicuna-13B example. Averaged over all models, BERTScore, an embedding similarity to the full-context answer, fell by 0.023.
- **LLMLingua** adds three parts. A **budget controller** keeps 85% of the instruction and 90% of the question, and drops whole demonstrations first, lowest perplexity first. **Iterative token-level compression** scores 100-token segments given the text already compressed, because a deletion changes how surprising the next token is. **Distribution alignment** instruction-tunes the small LM on text generated by the target LLM.



GSM8K on GPT-3.5-Turbo: the 2,366-token chain-of-thought prompt shrinks to 117 tokens (20×) while exact match moves from 78.85 to 77.33. Selective Context reaches 44.20 at 15×.

Jiang et al., "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," EMNLP 2023, arXiv:2310.05736v2, Figure 1; numbers from Table 2. Selective Context: Li et al., "Compressing Context to Enhance Inference Efficiency of Large Language Models," EMNLP 2023, arXiv:2310.06201v1.

What Compression Costs

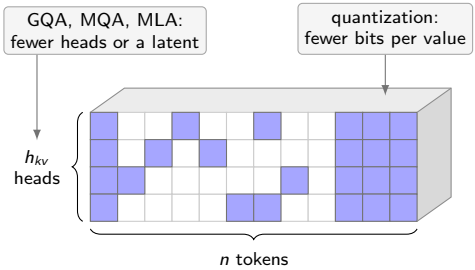
Method	Output	Needs	Question	Reported ratio	Reported effect
Selective Context	text	small LM	agnostic	about 2×	BERTScore −0.023
LLMLingua	text	small LM	agnostic	up to 20×	GSM8K 78.85 to 77.33
LongLLMLingua	text	small LM	aware	about 4×	NaturalQuestions 54.1 to 71.2
LLMLingua-2	text	small encoder	agnostic	2–5×	end to end 1.6–2.9× faster
Gist tokens	vectors	fine-tuned LLM	agnostic	26×	40% fewer FLOPs
ICAE	vectors	LoRA encoder	agnostic	4×	3.3× faster
AutoCompressors	vectors	fine-tuned LLM	agnostic	about 40×	4,096 tokens ≈ 512 plain

- **Exact strings go first.** LLMLingua's 20× GSM8K prompt reads "highl pens" and "36030lters", LongLLMLingua needs subsequence recovery to restore names, a gist model returned "Culture" for "Arts & Culture", and ICAE rebuilds random token strings at BLEU 0.2. Spans that must survive verbatim, such as code, identifiers and numbers, are safer left uncompressed.
- **Hard or soft.** Hard output is readable, works with any API and needs no change to the target model; the reported operating points are about 20× on redundant few-shot prompts and 2–5× on documents. Soft output packs more into each position (ICAE's 128 slots beat a 128-token GPT-4 summary), but it is tied to one model's weights, costs a training run and cannot be read.
- **Aware or agnostic.** Seeing the question lifted accuracy at 4× from 23.5 to 71.2 on the same test (answer in document 10), but the compressed prompt must then be rebuilt for every question. Query-agnostic output is computed once and cached.

Each ratio is the paper's own operating point on its own benchmark, so the rows are not directly comparable. Sources as on the previous slides.

Three Ways to Shrink the KV Cache

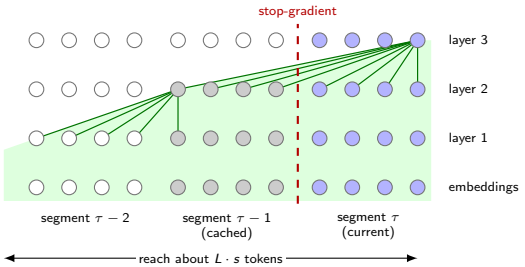
- The cache size, $2 \cdot L \cdot h_{kv} \cdot d_h \cdot (\text{bytes per value}) \cdot n$, is a product, and each factor can be attacked separately.
- **Fewer numbers per token:** GQA and MQA share key/value heads, and MLA caches one low-rank latent per token.
- **Fewer bits per number:** KV-cache quantization such as TurboQuant.
- **Fewer tokens: eviction** keeps a budget of $B < n$ tokens per head and layer and drops the rest, so memory and the attention cost of each decoding step scale with B.
- Each eviction method fixes which tokens to keep, when to decide (once after the prompt, or at every decoding step), and from which signal (position or attention). An evicted entry cannot come back later.



One layer's cache. Each of the 4 heads keeps its own $B = 6$ of the $n = 12$ tokens (blue); the grey boxes are the other two levers.

Segment-Level Recurrence

- Split n tokens into segments of s tokens and process them in order, carrying a memory of M vectors from one segment to the next. Attention then costs about $n\,(s + M)$ instead of n^2.
- **Cached states.** Layer ℓ of segment τ also attends to the layer-$(\ell - 1)$ states that segment $\tau - 1$ left behind. A **stop-gradient** makes this cache a constant in the backward pass, so no gradient enters the previous segment.
- **Reach.** Each layer looks one segment further back, so L layers reach about $L \cdot s$ tokens.
- Sliding-window attention with window w also reaches $L \cdot w$ through depth, but runs over the whole sequence in one pass. Here only one segment and its cache are held at a time.



Here $s = 4$ and $L = 3$. The top state of the last token in segment τ reads layer 2 of segments $\tau - 1$ and τ; the cached state it reaches in segment $\tau - 1$ had read layer 1 of segment $\tau - 2$. Shaded: everything that state can depend on.

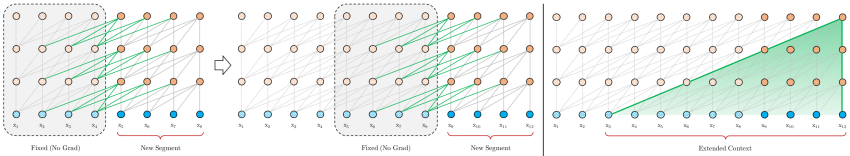
Transformer-XL: Reusing the Previous Segment

- Transformer-XL (Dai et al., ACL 2019) feeds every layer the cached states of the previous segment:

$$\tilde{H}_{\tau}^{\ell-1} = \left[\,\mathrm{SG}(H_{\tau-1}^{\ell-1}) \circ H_{\tau}^{\ell-1}\,\right], \qquad H_{\tau}^{\ell} = \mathrm{Layer}\big(H_{\tau}^{\ell-1} W_q,\ \tilde{H}_{\tau}^{\ell-1} W_k,\ \tilde{H}_{\tau}^{\ell-1} W_v\big).$$

  $H_{\tau}^{\ell} \in \mathbb{R}^{s \times d}$ holds the layer-ℓ states of segment τ (d: model width; H^0: token embeddings). SG is the stop-gradient and $\circ$ concatenates along the sequence, so the s queries of the segment meet $2s$ keys and values. W_q, W_k, W_v are the layer's projections; Layer is causal attention plus the feed-forward block. The cache can hold M states over several segments: $M = s$ in training, longer at evaluation.

- **Evaluation.** A fixed-window model recomputes a whole window for every predicted token. Reusing the cache is up to **1,874**× faster per token on enwik8 at an attention length of 3,800.



(a) Training phase. (b) Evaluation phase.

Dai et al., "Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context," ACL 2019, arXiv:1901.02860v3, Figure 2.

Transformer-XL: Relative Positions

- **Absolute positions break.** Every segment reuses positions $1, \ldots, s$, so token j of segment τ and token j of segment $\tau + 1$ get the same position vector, and the model cannot tell which came first.
- **Fix:** put the distance $i - j$ into every attention score, as RoPE and ALiBi also do. For query i and key j:

$$A_{i,j} = \underbrace{q_i^\top k_j}_{(a)} + \underbrace{q_i^\top W_{k,R}\, r_{i-j}}_{(b)} + \underbrace{u^\top k_j}_{(c)} + \underbrace{v^\top W_{k,R}\, r_{i-j}}_{(d)}$$

  (a) content addressing, (b) a position term that depends on the query's content, (c) a global content bias, (d) a global position bias. q_i and k_j come from content alone, r_{i-j} is a fixed sinusoidal encoding of the distance, $W_{k,R}$ is its key projection, and $u, v \in \mathbb{R}^d$ are learned vectors shared by all query positions.
- Fixed sinusoids let a model trained with a short memory use a longer one: on WikiText-103, 384 tokens of attention in training and 1,600 at evaluation.

Model (WikiText-103)	RECL
Transformer-XL, 151M	900
QRNN	500
LSTM	400
Transformer-XL, 128M	700
without recurrence	300
Vanilla Transformer	128

- **RECL** (relative effective context length, in words): grow the context until extra words cut perplexity by less than 1%, measured against the best model of the same group on the 10% hardest tokens. QRNN is a quasi-recurrent network.
- Hence the headline: 80% longer than RNNs and 450% longer than a vanilla Transformer.
- At publication: perplexity 18.3 on WikiText-103 (previous best 20.5) and 0.99 bits per character on enwik8.

Dai et al., arXiv:1901.02860v3, Section 3.3, Tables 1, 2 and 8, Appendix C.

What a Fixed-Size Memory Costs

- Every memory in this section has a fixed size, so the copying result applies.
- **Recall-heavy tasks** (table): Gated DeltaNet trails the Transformer by 6.4 points on average and by 28.5 on FDA. Attention layers whose 2K window spans these inputs close the gap.
- **Needles:** trained on 4K tokens, Gated DeltaNet finds a number hidden in essays 100.0% of the time at 1K tokens and 29.6% at 8K.
- **At scale:** from 410M to 7B parameters, MiniMax's lightning attention (a linear attention) matched softmax on common-sense tasks but fell short at needle retrieval, so its production model keeps one softmax layer in eight.
- **Independent check:** Hugging Face could not make Infini-attention reliable; quality fell as the number of memory compressions grew.

Accuracy (%), inputs truncated to 2K tokens

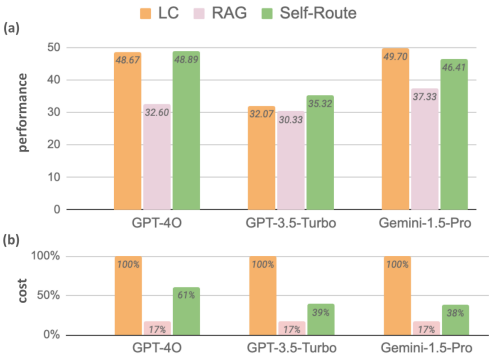
1.3B model	SWDE	FDA	Avg of 6
Mamba-2	19.1	25.3	29.8
DeltaNet	17.9	18.4	26.2
Gated DeltaNet	25.4	23.7	30.6
Llama-style Transformer	29.5	52.2	37.0
Gated DeltaNet + SWA	35.6	52.0	39.0

SWA: sliding-window attention layers, 2K window. SWDE: extract relations from web pages. FDA: retrieve key-value pairs from PDF documents. All models trained on the same 100B tokens. Yang, Kautz & Hatamizadeh, arXiv:2412.06464v3, Table 4; needles from Table 2.

MiniMax, "MiniMax-01: Scaling Foundation Models with Lightning Attention," technical report 2025, arXiv:2501.08313v1, Section 2.2 and Figure 7; Nguyen, von Werra & Wolf, "A Failed Experiment: Infini-Attention, and Why We Should Keep Trying?," Hugging Face blog, 14 Aug. 2024, huggingface.co/blog.

Self-Route: Let the Model Decide

- **Step 1, retrieve and route.** Give the model the query and the retrieved chunks (about 1.5k tokens) and allow it to reply "unanswerable". Answerable queries stop here.
- **Step 2, read everything.** Only the queries declared unanswerable go to the long-context model with the full input (10k to 100k tokens).
- The router is the model's own judgment of whether the chunks suffice, so no extra classifier is trained.
- Self-Route used 38% of the long-context cost with Gemini-1.5-Pro and 61% with GPT-4o, at average scores close to the long-context model (46.41 vs 49.70 and 48.89 vs 48.67).



"While long-context LLMs (LC) surpass RAG in long-context understanding, RAG is significantly more cost-efficient." Li et al., arXiv:2407.16833v2, Figure 1.

The Families Side by Side

Family	What shrinks	Exact recall of an old token	Training	Typical failure
Hard prompt compression	tokens read	no, once dropped	small compressor	numbers and names dropped
Soft compression	tokens become vectors	approximate	encoder or LLM fine-tune	exact strings blur
KV-cache eviction	cache to B entries	no, once evicted	none	needle evicted before the question
Fixed sparse patterns	n^2 to about $n\,w$	through global tokens and depth	pretraining or adaptation	pattern ignores content
Learned block selection	n^2 to about $n^2/b + n\,k\,b$	yes, if its block is chosen	native or continued training	selection misses
Segment recurrence	memory per segment	no, beyond the memory	training through segments	sequential, slower training
Linear-attention memory	a fixed-size state	no, capacity bound	pretraining	recall-heavy tasks
Retrieval	tokens read	only what is retrieved	a retriever	retrieval misses, multi-hop

- Production systems combine rows: sparse or linear layers for most of the depth, a few full-attention layers for exact recall, quantized caches, and retrieval in front.

A Decision Guide

1. **Measure first.** Probe the model at your real input lengths with RULER- or NoLiMa-style tasks built from your own data. The effective length bounds what any later technique can deliver, whatever the advertised window.

2. **Knowledge in a large or changing corpus:** retrieve, and send the full context only when the retrieved chunks cannot answer, as Self-Route does.

3. **One long document that must be read whole, with serving memory as the limit:** compress the cache with quantization and query-aware eviction. Keep the full prompt when follow-up questions are unknown in advance.

4. **An unbounded stream:** sink tokens plus a recent window keep generation fluent; add a recurrent memory when old content must be recalled.

5. **Training a model:** build length in with block-sparse attention or delta-rule layers, mixed with a few full-attention layers.

6. **Re-measure after every change** on the same probe. Compression and sparsity fail silently, and they fail first on the long-range cases.